

Contents

Guide 2 — The Recommendation Engine (TOPSIS), the simple version

1. The idea, in three sentences

2. How it works — five steps

3. The one formula to remember

4. Where each part comes from (the research)

5. A quick example

6. The files

7. Jury questions — ready answers

1

1

1

1

2

2

2

Guide 2 — The Recommendation Engine (TOPSIS), the simple version

How Mobile Match Algeria picks the best 3 plans for a user — in plain words, with the research each part comes from, and ready answers for the jury.

1. The idea, in three sentences

The user gives their **budget**, their **data need**, and **ranks** what matters most (price, data). The engine throws away every plan that doesn't fit the hard limits, then ranks the rest by **how close each plan is to the “ideal” plan**. It uses no AI and no other users' data, only this user's profile, so it works from day one.

2. How it works — five steps

1. **Filter**: drop any plan over budget, or with the wrong operator / type / network.

2. **Weights**: turn the user's ranking into numbers — price #1, data #2 → **0.75 / 0.25**.

3. **Compare to two reference plans**: the **ideal** (cheapest *and* most data) and the **worst** (priciest, least data).

4. **Closeness**: measure how near each plan is to the ideal versus the worst — a score from **0 (worst) to 1 (best)**.

5. **Rank by closeness**, show the **top 3** (the score becomes the match %).

3. The one formula to remember

closeness C = (distance to the worst plan)

(distance to the ideal) + (distance to the worst)

The “distance” is a plain **Euclidean (straight-line) distance** across the criteria. A plan sitting right on the ideal gets C = 1; one sitting on the worst gets C = 0.

4. Where each part comes from (the research)

Nothing here is invented — each step is a published method:

Part of the engine	What it does	Comes from (research)
Hard filters	remove plans that don't fit	standard constraint screening
Weights from a ranking	"price #1, data #2" → 0.75 / 0.25	Rank-Order Centroid — Barron & Barrett (1996)
Ranking by closeness to the ideal	the whole formula in §3	TOPSIS — Hwang & Yoon (1981); recent guide: Taherdoost & Madanchian (2023)

The only things that are *ours* (and they are not formulas): *which* criteria the user may rank (price, data), and that the **budget is a hard ceiling**. Everything that calculates a number is from the references above.

5. A quick example

Budget **2000 DA**, need **10 GB**, ranked **price #1, data #2** (weights 0.75 / 0.25). Three plans: **A** 1000 DA/8 GB, **B** 1100 DA/15 GB, **C** 600 DA/4 GB.

TOPSIS gives closeness **C ≈ 0.60, B ≈ 0.40, A ≈ 0.26** → **ranking C, B, A**. C wins (its low price is closest to the ideal under the 0.75 price weight); B beats A on data because they're close on price. Rank data first instead and B would rise.

6. The files

- [recommendation.ts](#) — the engine: filters, rocWeights(), topsis().
- [api/recommendations/route.ts](#) — the API (validated, rate-limited).
- [recommend/page.tsx](#) — the form + ranked cards (with savings).

7. Jury questions — ready answers

- **What are you calculating? A Euclidean distance?** → Yes. Each plan's straight-line distance to the ideal and the worst plan, combined into a closeness score (TOPSIS).
- **Did you invent the formula?** → No. The ranking is **TOPSIS** (Hwang & Yoon, 1981), the weights are **Rank-Order Centroid** (Barron & Barrett, 1996) — both published. Only the choice of criteria and the budget ceiling are ours.
- **Does it work with no users?** → Yes, it's content-based (uses plan attributes + the user's profile), so it has no cold-start problem.
- **Why TOPSIS, not AI / a weighted sum / lexicographic / AHP?** → AI needs a usage history we don't have; a weighted sum gives an unbounded, scale-sensitive score; strict lexicographic is too rigid; AHP needs many pairwise comparisons. TOPSIS is bounded, content-based, and pairs naturally with the ranking-based weights.
- **What if no plan fits?** → The list is empty (the budget ceiling is strict); the user can raise the budget or relax a filter.

Next guide: the search engine (token parser + FTS5). Tell me when you want it.